

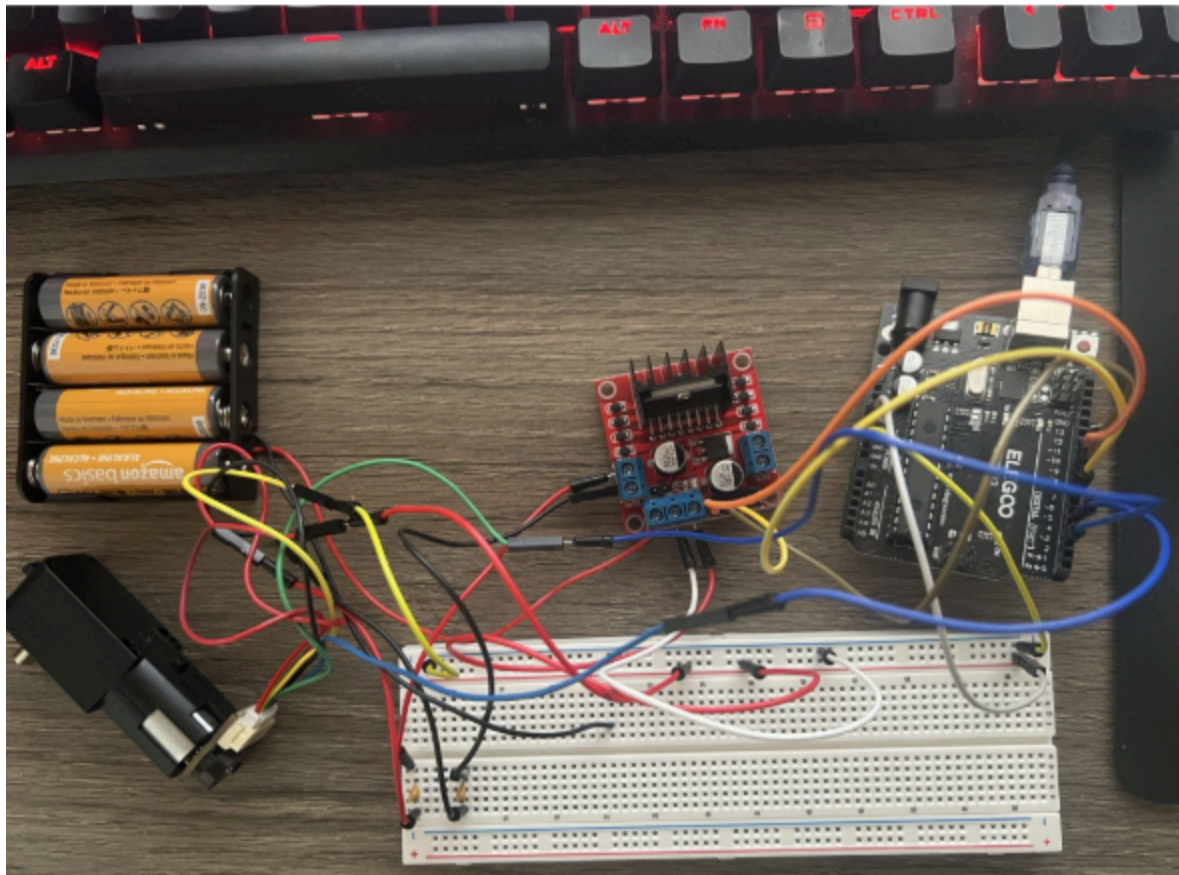
Embedded DC Motor Speed Controller

Platform: ATmega328P (Arduino UNO R3 board), PlatformIO. Code attached in the github repo

Timeline: September 4th, 2026 – Sept 15th, 2026

1. Summary

Implemented below is the designed, built and tested closed loop motor DC motor controller. Coded in bare-metal C, the Timer0 function serves as a system tick with a period of 1ms. Timer1 serves as a PWM generator, which acts as an external interrupt to count encoder edges, while the UART streams telemetry. Additionally, a PID loop is implemented to run such that every 200 ms there will be a regulator to set the motor to run at 60 RPM. Attached below is a picture of the setup.



Results:

Metric	Result
Setpoint	60 RPM (output shaft)
10–90% rise time	8.6 s, no startup overshoot
Entered $\pm 2\%$ band	~ 16.2 s
Steady-state error (pre-disturbance)	$\sim 1\%$ below setpoint
Peak PWM under drag	~ 220 and ~ 232 (of 255)
Minimum speed during strongest drag	~ 33 – 34 RPM
Recovery	Speed returned to the setpoint region after both disturbances; overshoot after release peaked at 65.6 and 67.5 RPM (see §6)

2. Objectives

- Expand my horizons from class, going from mostly MATLAB/Simulink to physical closed-loop systems that involve design and wiring with components in real life.
- Challenge myself to work with AVR C rather than use the arduino library

Requirements

- Regulate motor output speed to a fixed setpoint using encoder feedback
- Use hardware timers and interrupts rather than blocking delays or Arduino helper functions
- Provide serial telemetry for tuning and offline plotting
- Handle actuator limits (PWM saturation) and integrator windup

3. Hardware

Component	Details
MCU board	Arduino UNO R3 (ATmega328P, 16 MHz)
Motor	TT gear motor, 90:1 metal gearbox, 3–12 V

Component	Details
Encoder	12 PPR AB-phase Hall encoder on the motor shaft
Driver	L298N dual H-bridge breakout board
Motor supply	4 × AA battery pack [total 12 V]
Passives	100 nF ceramic capacitors [placed at motor power leads on breadboard]
Build	Breadboard + jumper wires

Pin map

Signal	Arduino pin	AVR pin	Notes
ENA (PWM)	D9	PB1 / OC1A	Timer1 hardware PWM
IN1	D8	PB0	Direction, held high
IN2	D7	PD7	Direction, held low
Encoder A	D2	PD2 / INT0	Rising-edge interrupt, internal pull-up
Encoder B	D4	PD4	Read inside ISR for direction, internal pull-up
UART TX	D1	PD1	Telemetry to PC

Figure 1. As-built breadboard setup: Arduino UNO, L298N driver, TT encoder motor, 4×AA pack.

(Figure 2, the schematic, goes here if you draw one.)

4. Firmware Architecture

All peripheral setup is done with direct register writes. [PlatformIO]

Peripheral	Configuration	Resulting behavior
Timer0	CTC mode, prescaler 64, OCR0A = 249, compare-match interrupt	16 MHz / 64 / 250 = 1 kHz tick; ISR increments a millisecond counter
Timer1	Fast PWM 8-bit (WGM 5), non-inverting on OC1A, prescaler 64	16 MHz / 64 / 256 ≈ 977 Hz PWM, duty 0–255
INT0	Rising-edge trigger on encoder channel A	ISR increments or decrements a volatile counter based on channel B
USART0	8N1, TX only, UBRR0 = 8	~115200 baud nominal (see note in Appendix A)
GPIO	PB0/PD7 outputs for L298N direction	Fixed forward direction

Program flow

- ☐ `main()` initializes peripherals, enables interrupts, sets forward direction, and starts with 0 duty.
 - ☐ A non-blocking loop checks the millisecond counter. Every 200 ms it runs one control step.
 - ☐ Shared variables (`encoderCount`, `millisCount`) are read inside short critical sections (interrupts disabled, SREG restored) to avoid torn reads of multi-byte values.
-

5. Control Design

5.1 Speed measurement

Speed is computed from the encoder count change over each control period:

$$\text{counts_per_rev} = \text{gear_ratio} \times \text{PPR} = 90 \times 12 = 1080 \quad (\text{per output-shaft revolution})$$

$$\text{RPM} = (\Delta \text{counts} / 1080) / (0.2 \text{ s} / 60)$$

Only channel A rising edges are counted ($\times 1$ decoding); channel B is sampled in the ISR to determine direction. At 60 RPM this yields 216 counts per 200 ms window, so one count corresponds to ≈ 0.28 RPM of measurement resolution.

5.2 PID law

Each 200 ms step:

$$\text{error} = \text{setpoint} - \text{rpm}$$

$$\text{integral} += \text{error} \times \text{dt} \quad (\text{clamped to } \pm 300)$$

$$\text{derivative} = (\text{error} - \text{prev_error}) / \text{dt}$$

$$\text{output} = K_p \cdot \text{error} + K_i \cdot \text{integral} + K_d \cdot \text{derivative}$$

$$\text{pwm} = \text{clamp}(\text{output}, 0, 255)$$

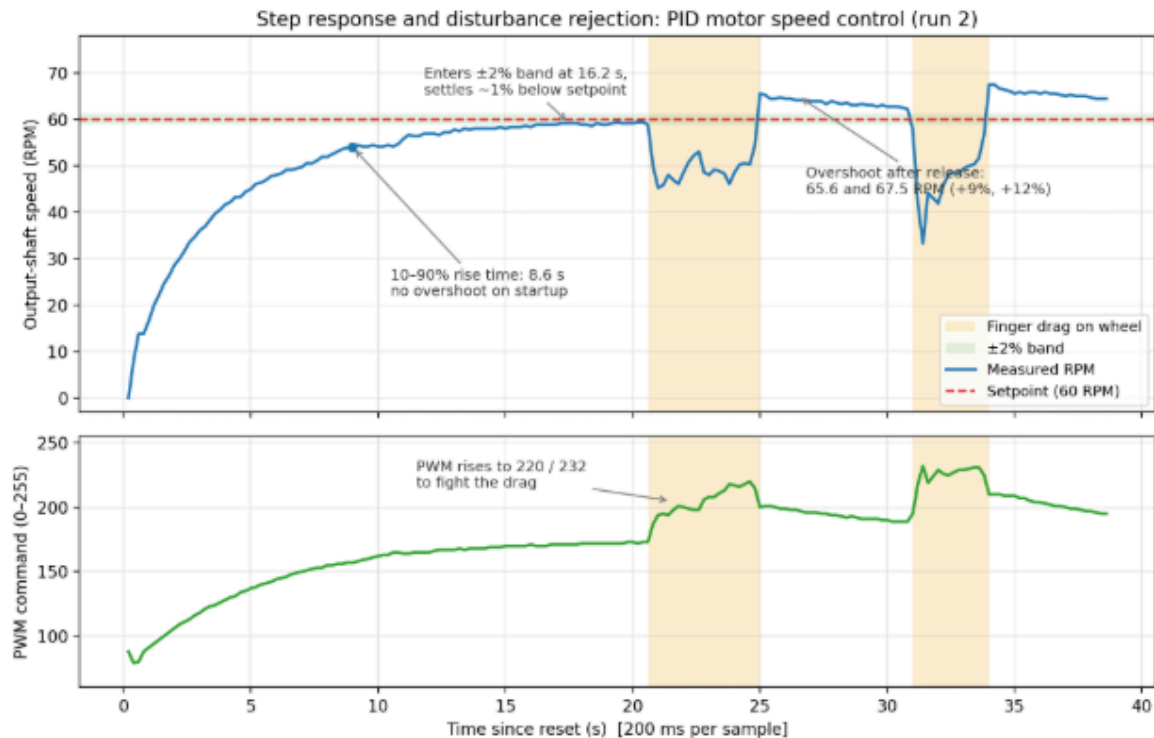
Parameter	Value
Kp	1.2
Ki	0.8
Kd	0.02
Control period	200 ms
Output range	0–255 (8-bit duty)
Integral clamp	± 300

- **Protections implemented:** output saturation and integral clamping.

- **Not implemented:** derivative filtering, feed-forward, conditional integration / back-calculation, bidirectional drive (output is clamped at 0).

5.3 Telemetry

Each control step prints **Setpoint**, **RPM**, and **PWM** over UART at 200 ms intervals.



6. Experimental Results

Figure 3. Step response and disturbance rejection, PID motor speed control (run 2). Top: measured output-shaft speed vs. 60 RPM setpoint with ±2% band. Bottom: PWM command. Orange regions mark manual finger drag on the wheel.

Step response

- ☐ 10–90% rise time: 8.6 s
- ☐ No overshoot
- ☐ Entered ±2% band at ~16.2 s and settled ~1% below setpoint

Disturbance rejection

	Disturbance 1	Disturbance 2
PWM during drag	rose to ~220	rose to ~232
Speed during drag	dropped substantially	dropped to ~33–34 RPM
After release	rose above setpoint, peak 65.6 RPM (+9%)	rose above setpoint, peak 67.5 RPM (+12%)

The controller increased PWM in response to load and regained the setpoint neighborhood after each disturbance. Drag was applied by hand, so the disturbances are **not repeatable or quantified in torque**; they demonstrate qualitative disturbance rejection, not a calibrated load test.

7. Analysis and Known Limitations

Post-release overshoot is integrator windup. During drag, the error stays large and the integral term accumulates. Since $K_i \times \text{clamp} = 0.8 \times 300 = 240$, the clamp allows the integral term alone to nearly saturate the output, while steady-state operation needed only ~170 PWM. When the drag is removed, the stored integral keeps PWM high (≈ 210 immediately after release), so speed overshoots. It then unwinds slowly, because a small negative error reduces the integral at only $K_i \times \text{error}$ per second. In the log, speed stays roughly 64–66 RPM after the second release rather than returning quickly to 60.

Slow rise time (8.6 s). The response is first-order-like with a modest K_p and a slow integral. Faster response is possible with higher gains, but that would worsen windup and noise sensitivity unless the fixes below are added.

Small steady-state offset (~1% below). Integral action should remove this over time; the run ended before full convergence. Plausible factors: slow integral action and encoder resolution (~ 0.28 RPM per count).

Possible actuator headroom limit. An L298N has a voltage drop of roughly 1.5–2 V, so a ~6 V pack leaves the motor with roughly 4 V at full duty. PWM near 230 under load suggests limited headroom.

Possible improvements:

1. Reduce the integral clamp so it cannot exceed the steady-state PWM need, or use conditional integration / back-calculation anti-windup
 2. Add a feed-forward term (approximate PWM per RPM) so the integrator only corrects the residual
 3. Reduce the control period (e.g., 20–50 ms) with a moving-average or low-pass filter on speed to get better dynamics without quantization noise
 4. Filter or drop the derivative term (it is small at $K_d = 0.02$ and adds noise sensitivity)
 5. Decode both encoder channels ($\times 2$ or $\times 4$) for finer speed resolution
 6. Use a higher-headroom supply and/or a MOSFET-based driver (e.g., lower voltage drop than the L298N)
 7. Repeat disturbance tests with a repeatable load (e.g., a friction brake or a second motor)
-

8. Build Log and Debugging

Date	Milestone
Sept 4	Parts ordered
Sept 6–7	Circuit assembled (first attempt Sept 6, completed Sept 7); began open-loop firmware
Sept 9	Open-loop firmware complete: motor spins, RPM reported over serial; began PID implementation
Sept 14	Closed-loop test failed: telemetry showed 0.00 RPM while PWM ramped to the 255 limit. Diagnosed as a hardware fault in the L298N driver board rather than a firmware or motor problem
Sept 14–15	Replaced the driver board; system operated correctly
Sept 15	Full closed-loop system functional; step and disturbance graphs recorded

Debugging summary

- **Symptom:** RPM stuck at 0.00 while the controller drove PWM to 255.
- **Firmware behavior:** the PID output and integral clamp behaved as designed (PWM saturated instead of running away), which confirmed the software logic was doing what it should when the plant did not respond.
- **Isolation:** I first suspected the motor jamming under load, then found I had been applying drag to the wrong component, which ruled the motor out. Replacing the L298N breakout board fixed the problem, pointing to the driver board.
- **Lesson:** when the feedback reads zero and the controller saturates, check the actuator path (driver, power, wiring) and the sensor independently before touching the control code.